

UML Y LOS DIAGRAMAS DE CLASES

1	Propósito de los diagramas de clases	1
2	Notación de los diagramas de clases	1
2.1	Clases, atributos y operaciones	1
2.2	Relaciones entre clases	4
2.3	Clases abstractas e interfaces	6
2.4	Objetos	8
2.5	Aspectos avanzados	8
3	Los diagramas de paquetes	10
4	Los diagramas de clases en los IDE	11
4.1	Elaboración de diagramas de clases en un IDE	11
4.2	Generación automática de código	12
4.3	Ingeniería inversa	12
5	Bibliografía	13

1 Propósito de los diagramas de clases

Los diagramas de clases son quizás los tipos de diagramas más fundamentales en UML. Se utilizan para capturar las relaciones estáticas del software; en otras palabras, cómo las cosas se ponen juntos.

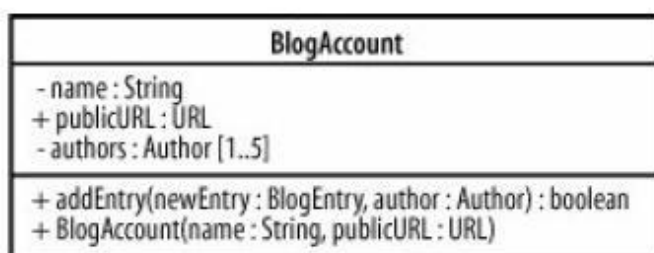
Al diseñar software se toman frecuentemente decisiones acerca de qué clases mantienen referencias a otras clases, que clases están compuestas de otras clases, etc. Las clases son en el corazón de cualquier sistema orientado a objetos; describen los diferentes tipos de objetos que el sistema puede tener.

Los diagramas de clases muestran las clases y sus relaciones. Proporcionan, pues, una forma de capturar la estructura lógica de un sistema.

2 Notación de los diagramas de clases

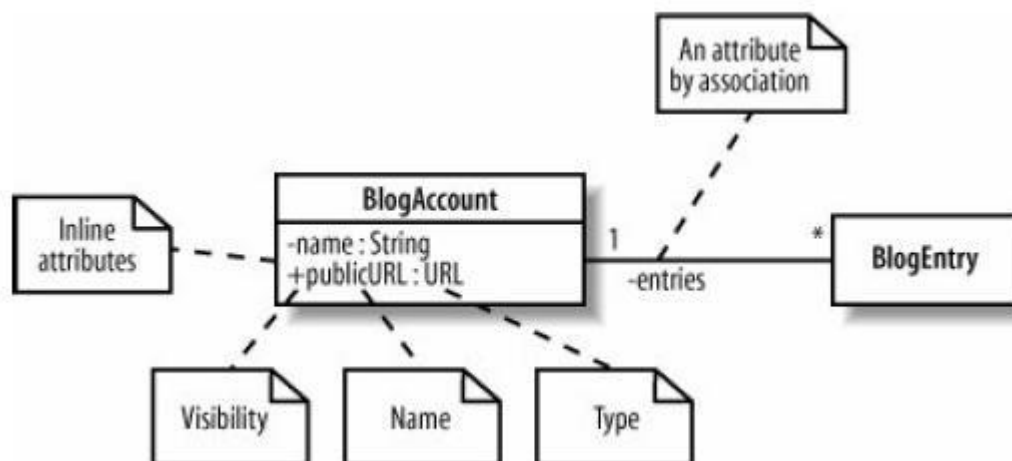
2.1 Clases, atributos y operaciones

En su forma más simple, una clase en UML se dibuja como un rectángulo dividido en un máximo de tres secciones. La sección superior contiene el nombre de la clase, la sección central contiene los atributos o la información que contiene la clase, y la última sección contiene las operaciones que representan el comportamiento de la clase. Las secciones de atributos y operaciones son opcionales.



Los atributos de una clase son los elementos de información que representan el estado de un objeto. Un atributo se puede representar en un diagrama de clases incluyéndolo dentro de la sección correspondiente del rectángulo de la clase (se denomina atributo en línea) o como una asociación con otra clase (se denomina atributo por asociación). Una regla útil afirma que clases “simples”, como la clase String de Java, e incluso clases de la biblioteca estándar, tales como la clase File en el paquete io de Java, generalmente se muestran mejor como atributos en línea.

En la declaración del atributo por lo general se incluye la visibilidad, el nombre y el tipo, aunque sólo el nombre es obligatorio.



Si esta clase se implementara en Java, se asemejaría a la siguiente:

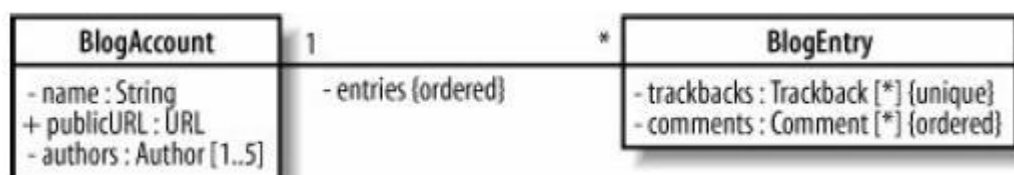
```

public class BlogAccount {
    private String name;
    public URL publicURL;
    private BlogEntry[] entries;
    // ...
}
  
```

Si un atributo tiene indicada la propiedad `{readOnly}` el valor del mismo no se podrá cambiar una vez que su valor inicial se ha establecido.

A veces un atributo representará más de un objeto. De hecho, un atributo podría representar cualquier número de objetos de su tipo; esto es como declarar que un atributo es un array. La multiplicidad permite especificar que un atributo representa en realidad una colección de objetos, y se puede aplicar a los atributos en línea y a los atributos por asociación.

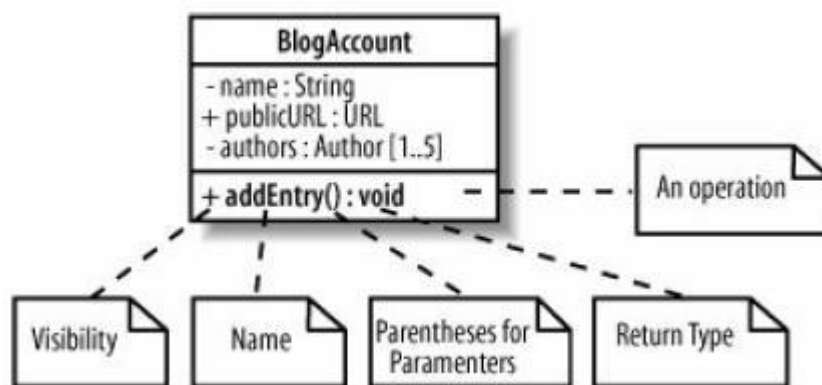
Por defecto, todos los atributos con multiplicidad son únicos; si se desea especificar que se permiten duplicados, entonces hay que especificar la propiedad `{not unique}`. Si importa el orden en que los objetos se almacenan en un atributo que tiene multiplicidad, entonces hay que indicar la propiedad `{ordered}`.



UML define tipos colección que combinan los posibles valores de las propiedades de unicidad y ordenación. En la tabla de la derecha se especifican, indicando en cursiva los valores por defecto.

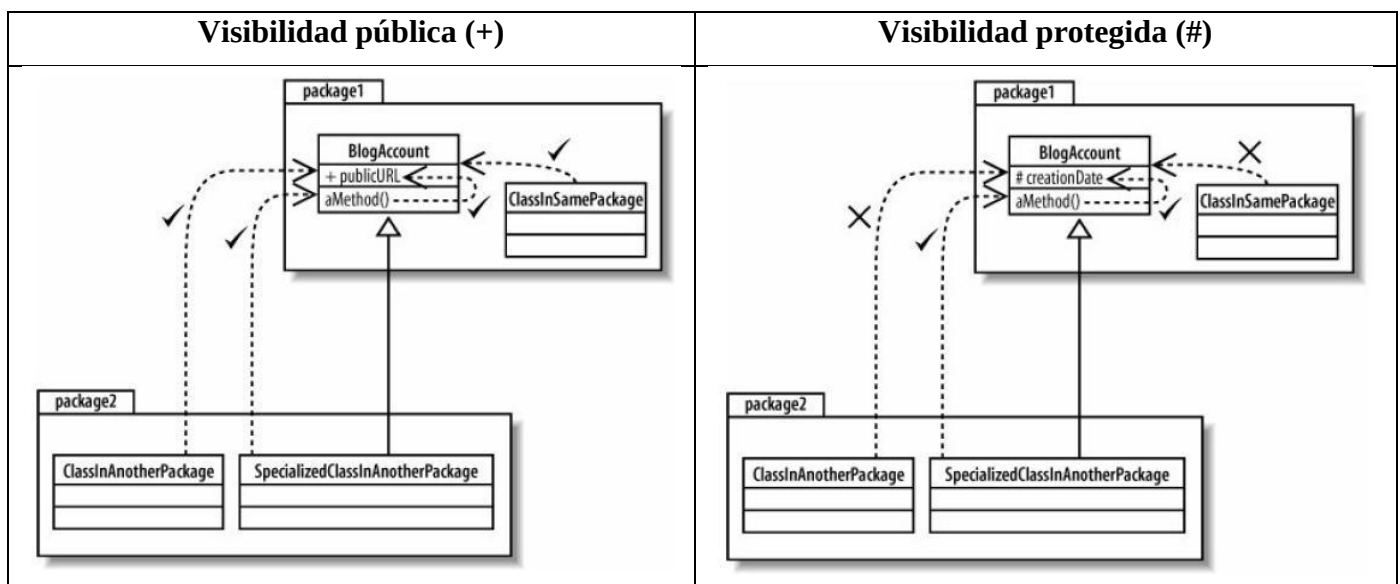
	Ordered	Not ordered
Unique	Orderedset	Set
Not unique	Sequence	Bag

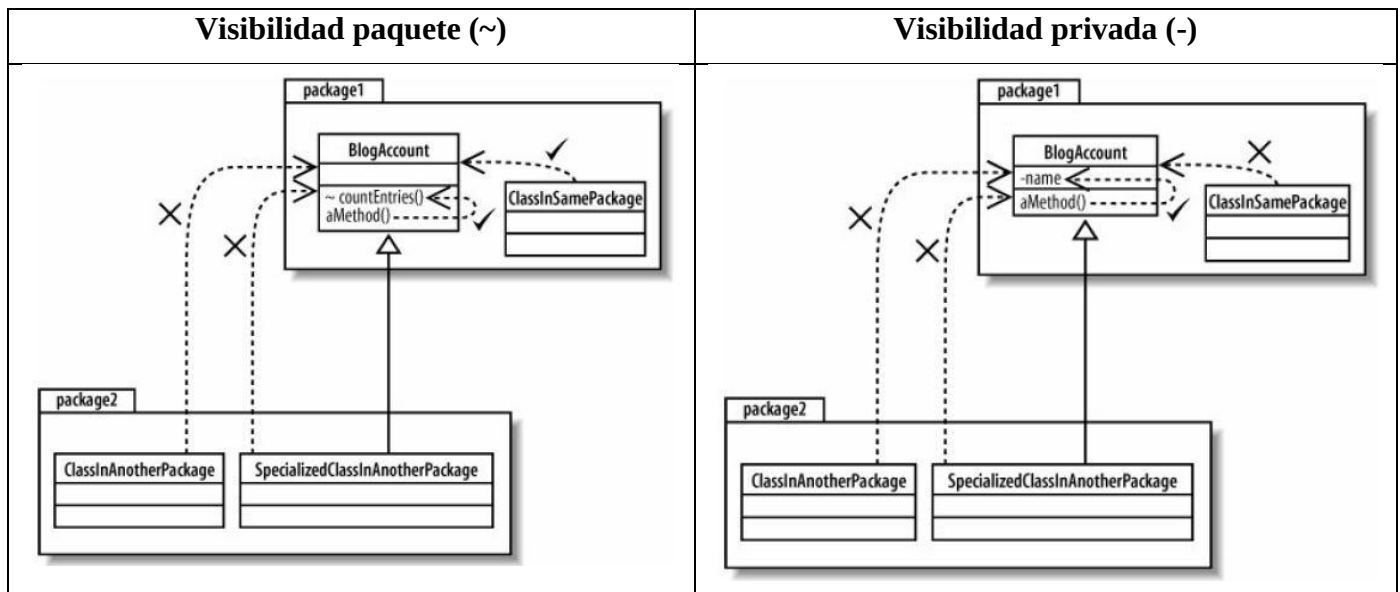
Las operaciones de una clase describen lo que una clase puede hacer, pero no necesariamente la forma en que va a hacerlo. Una operación es más como una promesa que declara que una clase contendrá algún comportamiento que hace lo que la operación dice que va a hacer. La colección de todas las operaciones que una clase contiene debe abarcar la totalidad del comportamiento de la clase, incluyendo todo el trabajo de mantener los atributos de la clase y, posiblemente, algún comportamiento adicional que está estrechamente asociado con la clase. La especificación de una operación en un diagrama de clases incluye la visibilidad, el nombre, un par de paréntesis entre los que se pueden indicar los parámetros que se necesitan para la operación, y un tipo de retorno.



Hay cuatro tipos diferentes de visibilidad que se pueden aplicar a los elementos de un modelo UML. Se utilizan para controlar el acceso a atributos, a operaciones, e incluso a veces a las propias clases. Son, ordenados de mayor a menor accesibilidad, los siguientes: pública, protegida, paquete y privada.

La colección de atributos y operaciones que se han declarado públicos en una clase constituyen la interfaz pública de la clase. La interfaz pública de una clase se compone de los atributos y las operaciones que pueden ser accedidos y utilizados por otras clases. Esto significa que la interfaz pública es la parte de una clase de la que más clases dependerán, por ello es importante que la interfaz pública de una clase cambie lo menos posible para evitar otros cambios allí donde se utiliza la clase.





En UML, las operaciones y los atributos (e incluso las propias clases) pueden ser declarados como estáticos: a veces se desea que todos los objetos de una clase en particular compartan la misma copia de un atributo; para ello el atributo ha de estar asociado con la clase misma y tendrá una vida útil más allá de la de los objetos que se creen como instancias de la clase; a estos atributos de clase se les denomina estáticos, y también a las operaciones de clase definidas para manipularlos. En UML los atributos y operaciones se declaran estáticas subrayándolas.

2.2 Relaciones entre clases

En un sistema software las clases no están aisladas sino que trabajan juntas utilizando diferentes tipos de relaciones. La fuerza de una relación entre clases se basa en el grado de dependencia que las clases involucradas en la relación tienen la una de la otra. Dos clases que son fuertemente dependientes una de otra se dice que están fuertemente acopladas; los cambios en una clase probablemente afectarán a la otra clase. El acoplamiento fuerte es por lo general, aunque no siempre, algo a evitar.

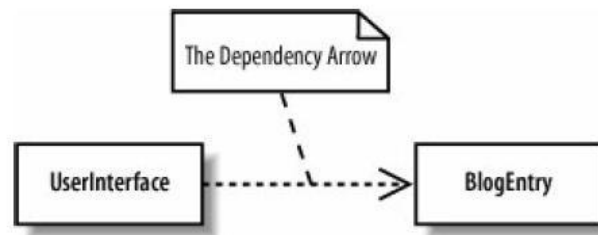
UML ofrece cinco tipos diferentes de relación entre clases:



2.2.1 Dependencia

Una dependencia entre dos clases declara que una clase tiene que saber acerca de otra clase para trabajar brevemente con los objetos de esa clase.

La relación de dependencia se utiliza a menudo cuando se tiene una clase que proporciona un conjunto de funciones de utilidad de propósito general, como en los paquetes de expresiones regulares de Java (`java.util.regex`) y de funciones matemáticas (`java.math`). Una clase depende de estas clases `java.util.regex` y `java.math` cuando utiliza las utilidades que éstas ofrecen.

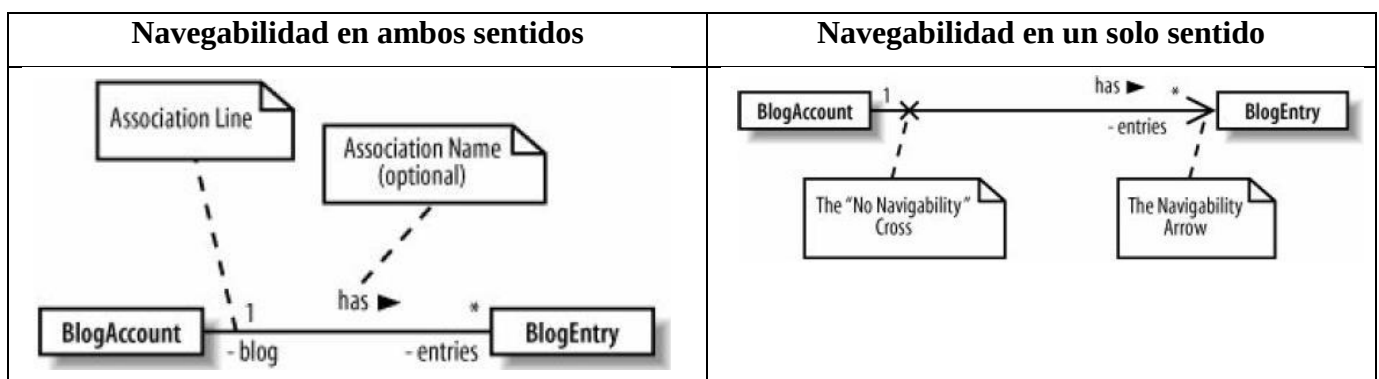


2.2.2 Asociación

Una relación de asociación significa que una clase realmente contendrá una referencia a un objeto u objetos de la otra clase, en forma de atributo. Cuando objetos de una clase trabajan con objetos de otra clase durante un tiempo prolongado, entonces la relación entre las clases es una firme candidata a ser una asociación en lugar de sólo una dependencia. Las asociaciones se suelen poder leer como "... tiene...".

Las líneas de vida de dos objetos unidos por asociaciones no suelen estar unidas entre sí (es decir, uno puede ser destruido sin que esto implique la destrucción del otro).

En una relación de asociación se ha de especificar la navegabilidad para describir la clase que contiene el atributo que soporta la relación. Una relación puede ser navegable en ambos sentidos, pero es más común que sólo una clase haga referencia a la otra en una asociación.



La implementación de estas clases en Java, se podría hacer así:

```

public class BlogAccount {
    private BlogEntry[] entries;
    ...
}
public class BlogEntry {
    private BlogAccount blog;
    ...
}
  
```

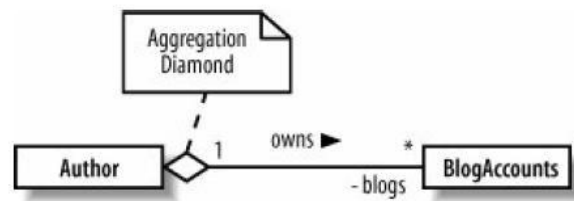
```

public class BlogAccount {
    private BlogEntry[] entries;
    ...
}
public class BlogEntry {
    ...
}
  
```

2.2.3 Agregación

La agregación es en realidad una relación más fuerte que la asociación y se utiliza para indicar que una clase realmente posee, pero puede compartir, los objetos de otra clase. Normalmente implica la propiedad y puede implicar una relación entre las líneas de vida. Una agregación generalmente se lee como "... posee ...".

La implementación del código Java para una relación de agregación es exactamente la misma que la de una relación de asociación; resulta en la introducción de un atributo.

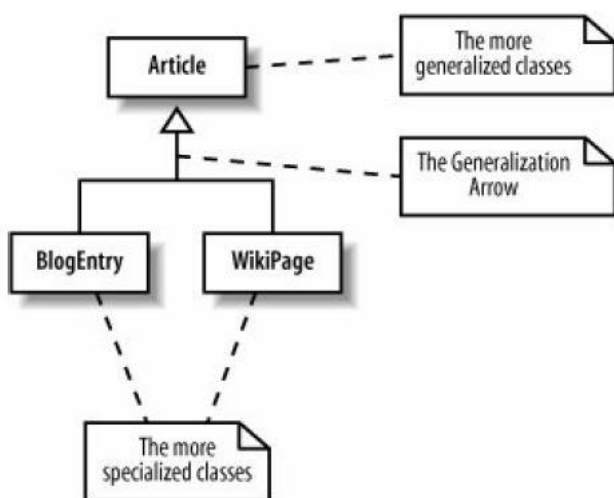


2.2.4 Composición

La composición representa una relación muy fuerte entre las clases, hasta el punto de modelar las partes internas que componen una clase. La composición se utiliza para capturar una relación todo-parte.

La pieza “parte” de la relación sólo puede estar implicada en una relación de composición en cualquier momento dado. La duración de las instancias que intervienen en las relaciones de composición está casi siempre ligada; si la instancia propietaria es destruida, casi siempre se destruye la instancia “parte”. Una relación de composición se lee habitualmente como “... es parte de ...”.

De manera similar a la agregación, la implementación del código Java para una relación de composición se traduce sólo en la introducción de un atributo.



2.2.5 Generalización

La generalización se utiliza para describir una clase que es un tipo de otra clase.

La clase más generalizada de la que se hereda es referida a menudo como padre, base o superclase; las clases especializadas son denominadas hijas o clases derivadas. Las clases especializadas heredan todos los atributos y métodos que son declarados en la clase generalizada y pueden añadir operaciones y atributos que son sólo aplicables en casos especializados.

La clave de por qué la herencia se denomina generalización en UML está en la diferencia entre lo que representan una superclase y una clase hija. Una

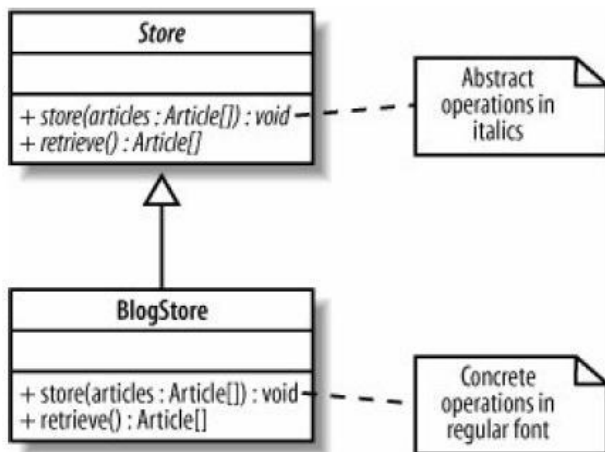
superclase describe un tipo más general, que se especializa en las clases hijas.

Las generalizaciones pueden ser leídas como “... es un ...”, a partir de la clase más específica y hacia la clase general.

2.3 Clases abstractas e interfaces

2.3.1 Clases abstractas

Las clases abstractas son un mecanismo muy poderoso que permiten definir comportamientos y atributos comunes, pero dejan algunos aspectos de cómo una clase trabajará a subclases más concretas.



A veces, cuando se utiliza la generalización para declarar una clase reutilizable y genérica, no se puede implementar todo el comportamiento que necesita la clase general. En UML para indicar que la implementación de esas operaciones se va a dejar a las subclases se han de declarar esas operaciones como abstractas, escribiendo su *signatura* en cursiva.

Si cualquier operación de una clase es declarada abstracta, la clase en sí también debe ser declarada como abstracta, escribiendo su nombre en cursiva. Una clase abstracta no puede ser instanciada como un objeto, porque faltan partes de la definición de clase: las partes abstractas. Las clases hijas de la clase

abstracta se pueden instanciar como objetos si completan todas las operaciones abstractas que le faltan a su superclase, convirtiéndose así en clases concretas.

2.3.2 Interfaces

Una interfaz es una colección de operaciones que no tienen las correspondientes implementaciones de los métodos; es similar a una clase abstracta que contiene sólo métodos abstractos.

Las interfaces tienden a ser mucho más seguras que las clases abstractas, ya que evitan muchos de los problemas asociados con la herencia múltiple. Esta es la razón por la que lenguajes de programación como Java permiten a una clase implementar cualquier número de interfaces, pero sólo heredar de una clase.

Notación “estereotipo”	Notación “pelota”
<pre> classDiagram class EmailSystem { <<interface>> +send(message : Message) : void } </pre>	<pre> classDiagram class EmailSystem { <<interface>> +send(message : Message) : void } </pre>
<pre> classDiagram class EmailSystem { <<interface>> +send(message : Message) : void } class SMTPMailSystem { +send(message : Message) : void } EmailSystem < .. SMTPMailSystem </pre>	<pre> classDiagram class EmailSystem { <<interface>> +send(message : Message) : void } class SMTPMailSystem { +send(message : Message) : void } EmailSystem < .. SMTPMailSystem </pre>
Ejemplo:	Ejemplo:
<pre> classDiagram class Sortable { <<interface>> +comesBefore(object : Sortable) : boolean } class Person { +comesBefore(object : Sortable) : boolean } Sortable < .. Person </pre>	<pre> classDiagram class Sortable { <<interface>> } class Person { } class Alphabetizer { } Sortable < .. Person Sortable < .. Alphabetizer </pre>

2.4 Objetos

Un objeto es una instancia de una clase. Se pueden tener varias instancias de una clase llamada Coche: un coche rojo de dos puertas, uno azul de cuatro puertas, y un coche verde ranchera. Cada instancia de Coche es un objeto y se le puede dar su propio nombre, aunque es común ver objetos sin nombre, o anónimos, en los diagramas de objetos. Típicamente se muestra el nombre del objeto seguido de dos puntos y de su clase, y se indica que se trata de una instancia de una clase, subrayando el nombre y la clase.

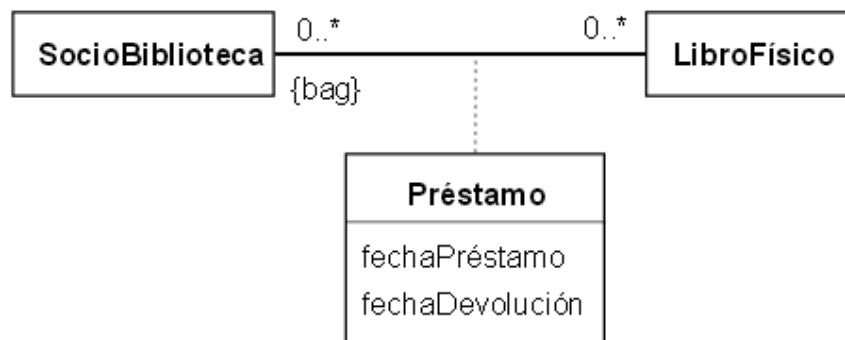


2.5 Aspectos avanzados

2.5.1 Clases asociación

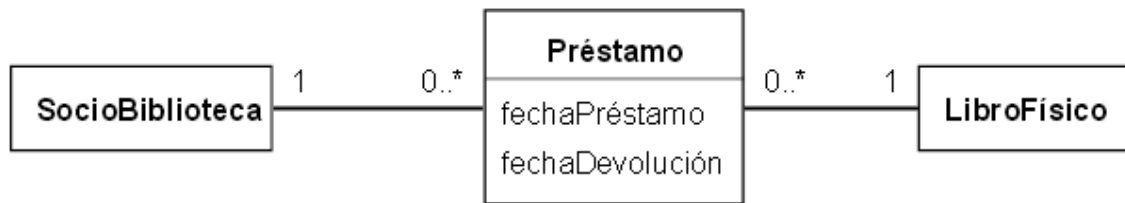
Cuando una asociación tiene propiedades propias (y/o comportamientos) se representa como una clase unida a la línea de la asociación por medio de un segmento discontinuo. Tanto la línea como el rectángulo de la clase representan el mismo elemento conceptual, la asociación, por tanto ambos tienen el mismo nombre, el de la asociación. Cuando la clase asociación sólo tiene atributos el nombre suele ponerse sobre la línea; por el contrario, cuando la clase asociación tiene alguna operación o asociación propia, entonces se escribe el nombre en la clase asociación y se suele quitar de la línea.

Así pues, una clase asociación representa los atributos y operaciones que sólo existen porque la asociación existe: no están vinculados a los objetos de cada extremo de la asociación. El principal beneficio obtenido al usar una clase asociación es que incorpora la restricción de que cada instancia de la clase asociación sólo puede caracterizar una asociación concreta entre cualesquiera dos objetos participantes.



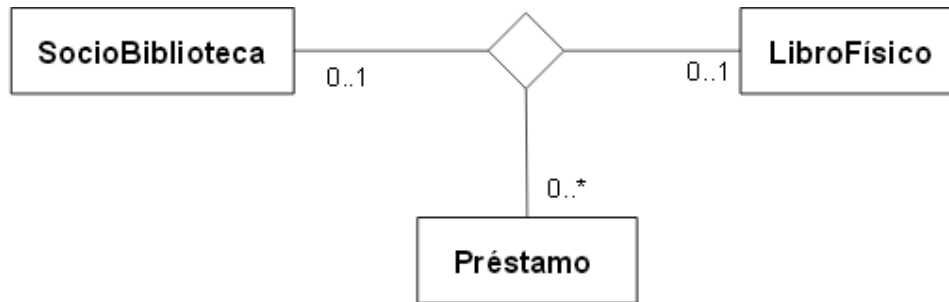
Las clases asociación son especialmente útiles durante el análisis. La mayoría de los lenguajes de programación no las soportan directamente, por lo tanto durante el diseño se podrá decidir una implementación concreta de entre las posibles. Sin embargo, el concepto fundamental de la clase asociación no cambia.

Cuando una asociación con clase asociación se traduce a código, a menudo resultan tres clases: una para cada extremo de la asociación y otra para la propia clase asociación. Puede no haber un vínculo directo entre los extremos de la asociación, pues el diseñador podría requerir pasar por la clase asociación para llegar al otro extremo del enlace. En otras palabras, `SocioBiblioteca` podría no tener una referencia directa a `LibroFísico`, sino tenerla a `Préstamo`. `Préstamo` tendría entonces una referencia a `LibroFísico`.



2.5.2 Asociaciones n-arias

En el caso de una asociación en la que participan más de dos clases, las clases se unen con una línea a un diamante central; además, como en las asociaciones binarias, puede precisarse de una clase asociación.



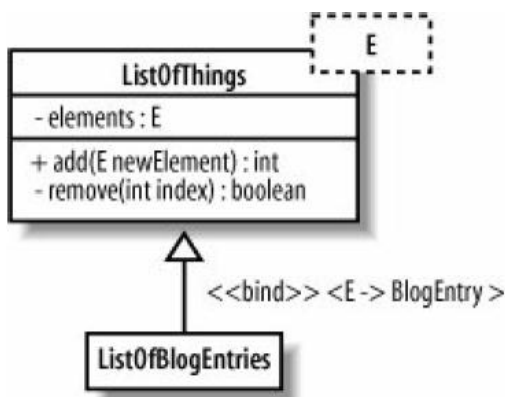
Si consideramos el ejemplo, la multiplicidad en Préstamo indica con cuántas parejas de SocioBiblioteca + LibroFísico puede estar asociada cada instancia de Préstamo.

Lo que no se restringe en el diagrama es en cuántas instancias de la asociación puede participar cada instancia de Préstamo: sería legal, por ejemplo, que el mismo préstamo estuviera asociado con dos libros diferentes, y cada uno de ellos con un socio distinto. Por tanto, la restricción consistente en que cada préstamo sólo pueda estar asociado con un socio y un libro, de ser necesaria, habrá de especificarse expresamente: una restricción se escribe entre llaves a continuación del elemento afectado o dentro de una anotación enlazada al mismo; será una expresión booleana que se podrá definir usando un lenguaje formal como OCL (*Object Constraint Language* – Lenguaje de Restricción de Objetos) u otro lenguaje de programación, o incluso el lenguaje natural.

2.5.3 Clases parametrizables

UML permite proporcionar abstracciones para el tipo de clase con el que una clase puede interactuar.

Por ejemplo, se podría escribir una clase colección que pudiera contener objetos de cualquier tipo (en Java probablemente sería una colección de objetos `Object`). Sin embargo, aunque la clase colección pudiera ser capaz de soportar cualquier tipo de objeto, seguramente interesaría que todos los objetos de una colección dada sean del mismo tipo. UML permite crear y especificar este tipo de abstracciones utilizando clases parametrizables.



Se puede indicar que una clase es parametrizable (también llamada plantilla, o *template* en inglés) dibujando un rectángulo de trazos en la esquina superior derecha de la clase. Para cada parámetro de la plantilla es necesario especificar en el rectángulo un nombre que actuará como marcador de posición para el tipo real. Un ejemplo de una clase colección que puede soportar cualquier tipo de objeto sería `ListOfThings`.

Para utilizar la clase parametrizable `ListOfThings`, primero hay que concretar su parámetro; se necesita especificar la clase real con la que la plantilla va a trabajar: La clase

`ListOfBlogEntries` reutilizará la totalidad del comportamiento genérico de la clase `ListOfThings`, restringiendo ese comportamiento a sólo añadir y eliminar objetos `BlogEntry`.

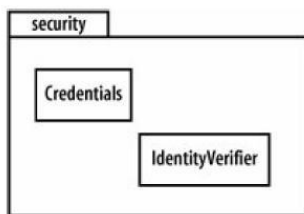
También se puede indicar la clase concreta de un objeto que es una instancia de una clase parametrizable, con la siguiente notación:

`listOfBlogEntries : ListOfThings <E-> BlogEntry`

3 Los diagramas de paquetes

Cuando un programa de software crece en complejidad, puede contener fácilmente cientos de clases. Una forma de imponer una estructura es organizar las clases en grupos relacionados lógicamente. Las clases que se ocupan de la interfaz de usuario de la aplicación pueden pertenecer a un grupo, las clases de utilidad pueden pertenecer a otro, etc.

En UML, los grupos de clases son modelados con paquetes. La mayoría de los lenguajes orientados a objetos tienen alguna construcción análoga a los paquetes UML para organizar las clases y evitar conflictos de nombres entre ellas. Por ejemplo, Java tiene paquetes.



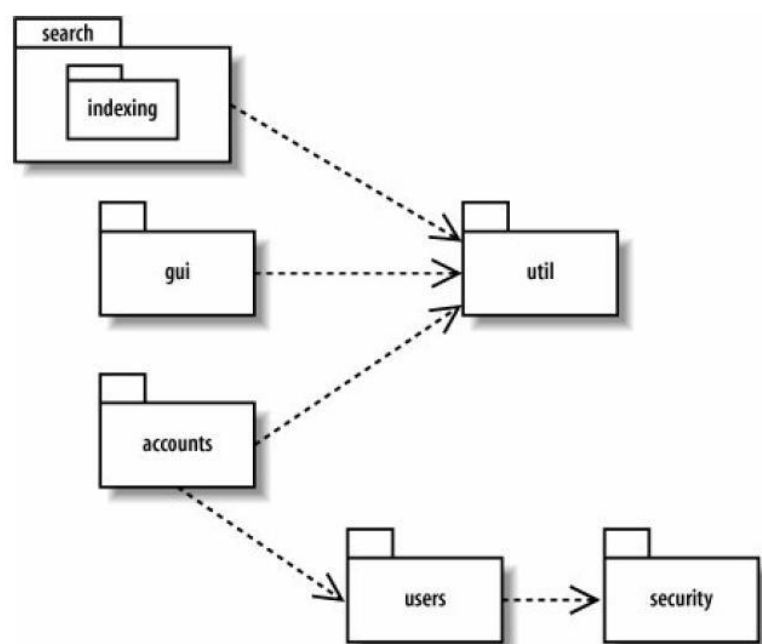
Los paquetes UML pueden organizar casi cualquier elemento UML; por ejemplo, son comúnmente utilizados para agrupar casos de uso; no obstante, su uso más común es la de organizar las clases en los diagramas de clases.

A veces una clase de un paquete tiene que utilizar otra clase de otro paquete. Esto provoca una dependencia entre paquetes: si un elemento en el paquete A utiliza un elemento en el paquete B, entonces el paquete A depende del

paquete B. Los diagramas de paquetes se utilizan a menudo para ver las dependencias entre paquetes. Dado que un paquete se puede “romper” si otro paquete del que depende cambia, comprender las dependencias entre los paquetes es vital para la estabilidad del software.



Un diagrama de paquetes típico podría ser el siguiente:



4 Los diagramas de clases en los IDE

Hoy en día existen cientos de herramientas CASE que soportan el lenguaje UML y por lo tanto es imposible encontrar ninguna página que las compare todas. En la página web siguiente se proporcionan algunas listas clasificadas de herramientas UML que pueden servir para empezar a buscar la más adecuada a los diferentes intereses: <http://modeling-languages.com/herramientas-para-uml/>

Una comparativa muy completa se puede encontrar en la Wikipedia:

http://en.wikipedia.org/wiki/Comparison_of_Unified_Modeling_Language_tools

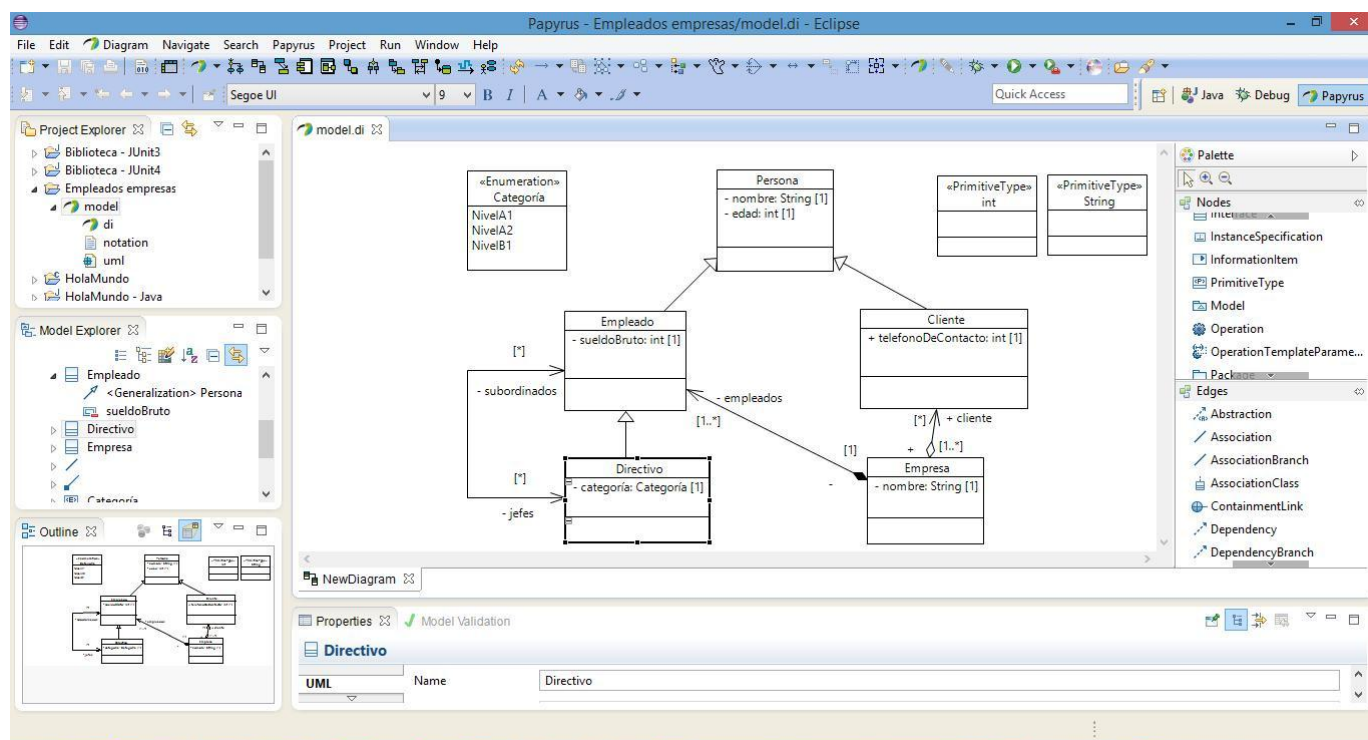
Entre las herramientas libres podemos encontrar StarUML, ArgoUML o UMLet, Umbrello UML Modeller y entre las comerciales BOUML, UModel, MagicDraw, Visual Paradigm for UML o Microsoft Visio.

Por otra parte, Eclipse incorpora numerosos plugins que incorporan los diagramas UML al IDE: UML Designer, Papyrus, UMLet, etc.

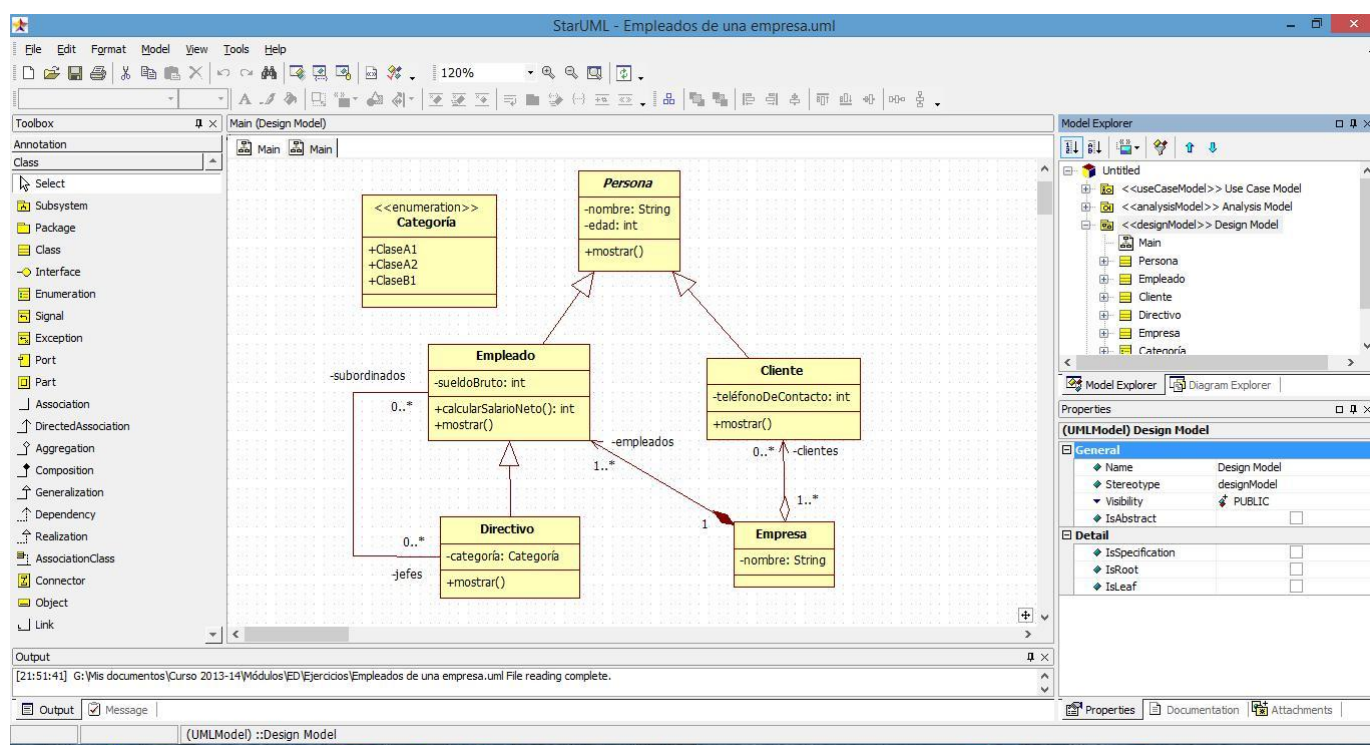
Existen herramientas gráficas, herramientas textuales (que permiten la especificación textual de modelos UML y que automáticamente dibujan el diagrama UML correspondiente), herramientas para UML ejecutable (buscan definir los modelos UML con suficiente precisión como para permitir su ejecución directa, eliminando incluso la necesidad de programar), etc.

4.1 Elaboración de diagramas de clases en un IDE

Utilización del Plugin Papyrus de Eclipse:

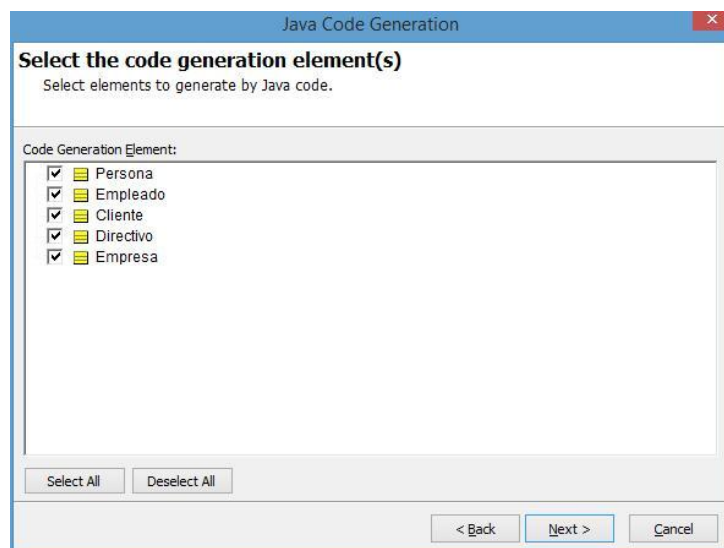
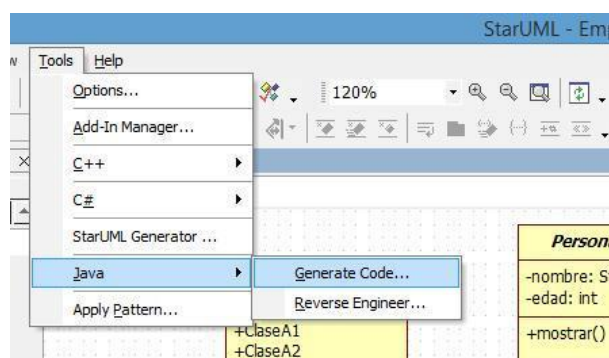


Utilización de StarUML:



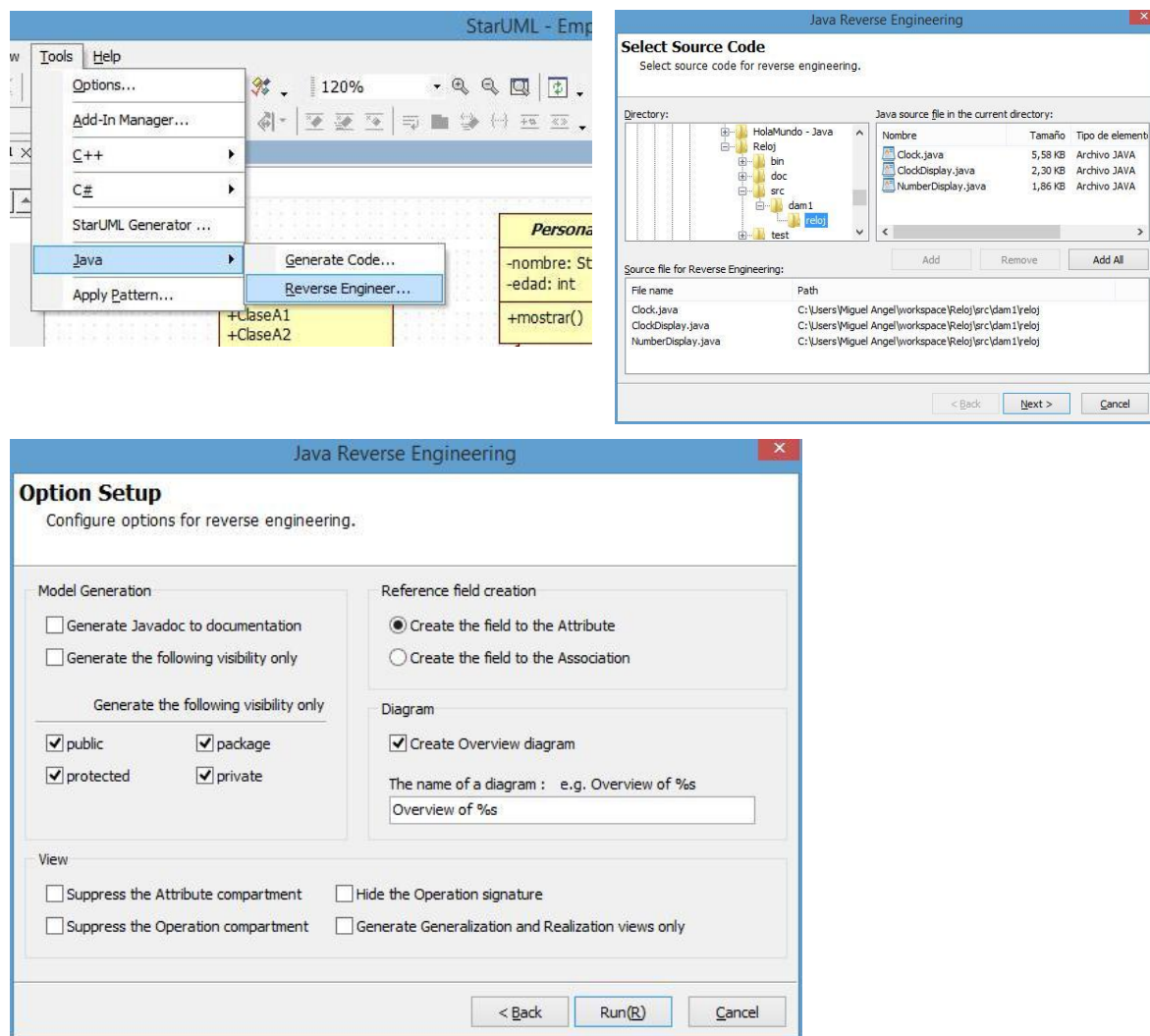
4.2 Generación automática de código

Algunos IDEs soportan la generación automática de código fuente a partir de los diagramas UML:



4.3 Ingeniería inversa

Algunos IDEs soportan la generación automática de los diagramas UML a partir de código fuente:



5 Bibliografía

- HAMILTON, K.; MILES, R. (2006): *Learning UML 2.0*. O'Reilly
- PILONE, D.; PITMAN, N. (2005): *UML 2.0 in a Nutshell*. O'Reilly
- O'DOCHERTY, M. (2005): *Object-oriented analysis and design: understanding system development with UML 2.0*. England. John Wiley & Sons
- “Desarrollo orientado a objetos con UML”, CECyT “Juan de Dios Bátiz Paredes” – IPN
- ZAPATA, M^a A.: “Diseño Estructural: Diagramas de Clases” del curso “Modelos Matemáticos en Bases de Datos/Métodos Matemáticos en Ingeniería del Software”. OCW – Universidad de Zaragoza.
- <http://www.sewo.biz/UML2/UML2Intro.php>
- <http://modeling-languages.com/herramientas-para-uml/>
- http://en.wikipedia.org/wiki/Comparison_of_Unified_Modeling_Language_tools